



Pulse — Full-Stack MERN Interview & Presentation Guide

Project Name: Pulse — Modern Social Media Platform

Tech Stack: MongoDB Atlas Cloud, Express.js, React 18 (Vite), Node.js (MERN)

MongoDB Atlas

Express 4

React 18

Node.js

JWT Auth

Multer Uploads

1. 🚦 30-Second Elevator Pitch (What to say first)

"Pulse is a modern, responsive full-stack social media web application built using the MERN stack (MongoDB Atlas, Express, React 18, Node.js). It features JWT-based secure authentication, real-time-style feed generation based on user follow graphs, photo attachment file uploads using Multer, post interactions (likes, comments), a notification system, and an automated cloud database seeding pipeline. The application also includes an automated fallback architecture to ensure 100% uptime even during network or database connectivity disruptions."

2. 🏢 System Architecture & Data Flow

CLIENT LAYER (React 18 + Vite)

- React Router 6 Navigation
- AuthContext (Global User State & Token Management)
- Axios Centralized API Client with 401 Interceptors

| HTTP / REST API Calls (JSON)



SERVER LAYER (Express.js / Node.js)

- CORS & JSON Body Parser Middleware
- JWT Authentication Middleware (`Bearer <token>`)
- Route Handlers: /api/auth, /api/posts, /api/users, /api/upload
- Multer File Storage Engine (`/uploads` static hosting)

| Mongoose ORM / Driver



DATABASE LAYER (MongoDB Atlas Cloud)

- User Schema (Credentials, Bio, Followers/Following)
- Post Schema (Author Ref, Content, Image URL, Likes, Comments)
- Notification Schema (Sender, Recipient, Event Type)

3. 🗝️ Tech Stack Breakdown

Layer	Component	Purpose / Why Selected
Frontend	React 18	Component-driven UI, fast reconciliation with Virtual DOM, Hooks (`useState`, `useEffect`, `useRef`).
Build Tool	Vite 5	Lightning-fast HMR (Hot Module Replacement) and optimized production bundles.
Styling	Vanilla CSS3	Custom design system with CSS variables, dark mode glassmorphism aesthetics.
Backend	Node.js + Express 4	Non-blocking asynchronous I/O event loop, lightweight modular routing.
Database	MongoDB Atlas	Cloud NoSQL document database providing dynamic schema flexibility and index optimizations.
ORM	Mongoose 8	Strongly-typed ODM for schema validation, population (`.populate()`), and JSON data transformation.
Auth	JWT + bcryptjs	Stateless authorization using 7-day signed tokens; salted password hashing (10 rounds).
FileUpload	Multer	Multipart form-data handling with mime-type filters and 5MB size limits.

4. 🎯 3-Minute Demo Walkthrough Script for Your Interview

- Login Page (/login)**: Show the dark-mode interface. Explain JWT storage in `localStorage` and header injection via Axios. Log in using sample credentials: `alex@example.com` / `password123`.
- Home Feed (/)**: Show the dynamic feed populated with posts from followed users (`sarah_design` , `tech_daily`). Click "Photo", upload an image, and submit a post. Click the Like (Heart) button.
- Explore & User Discovery (/explore)**: Navigate to Explore to show global posts. Use the Search Bar on the right panel to search for users (`sarah`). Click Follow / Unfollow.
- Profile Page (/profile/:username)**: Click on a user's avatar to view their public profile page with total followers, following count, and post history.
- Notifications (/notifications)**: Show notifications created when other users liked or commented on posts.

? Top 12 Technical Interview Questions & Model Answers

● Section 1: System Design & Architecture

Q1: Why did you choose the MERN stack for this project?

Model Answer: MERN allows a unified JavaScript/Node ecosystem across both client and server, reducing context switching. React 18 provides high-performance client rendering, Express delivers lightweight REST APIs, and MongoDB offers flexible document modeling ideal for social media data (posts with embedded comments and dynamic arrays for likes).

Q2: How does the application handle state management?

Model Answer: We used React Context API (`AuthContext`) for global authentication state (user object, login/logout functions, token validation). Page-level and component-level state are managed using React `useState` and `useEffect` hooks to keep components modular.

Q3: How do you handle file uploads in this architecture?

Model Answer: File uploads are handled using `Multer` middleware on Express. The client sends `multipart/form-data` via Axios to `/api/upload`. Multer validates file mime-types (images only), applies a 5MB limit, generates unique file names, saves to `server/uploads/`, and returns the static image URL for MongoDB storage.

● Section 2: Database & Mongoose Modeling

Q4: Explain your MongoDB Schema design for User and Post.

Model Answer: The `User` schema contains basic fields (`username` , `email` , `password` , `bio`) along with array references of ObjectIds pointing to `User` for `followers` and `following` . The `Post` schema contains an `author` reference pointing to `User` , string `content` , `image` URL, an array of `likes` ObjectIds, and embedded `comments` subdocuments.

Q5: How do you query posts for the Home Feed vs Explore Feed?

Model Answer: For the Home Feed, we query posts where `author` matches `[...me.following, me._id]` sorted by `createdAt: -1` with `.populate('author')` . For Explore, we query all posts sorted by `createdAt: -1` .

Q6: What is Mongoose Population (`.populate()`) and why is it used here?

Model Answer: References between MongoDB collections are stored as ObjectIds. `.populate('author', 'username name avatar')` performs a server-side join to fetch referenced User details without returning sensitive fields like password hashes.

● Section 3: Authentication & Security

Q7: How is authentication implemented in this app?

Model Answer: We use JSON Web Tokens (JWT). Upon successful login, the server hashes passwords using `bcryptjs` (10 rounds) and generates a signed JWT containing `userId` with a 7-day expiration. The client stores it in `localStorage` and passes it in the `Authorization: Bearer <token>` header.

Q8: How do you protect private routes on the frontend?

Model Answer: We created a `ProtectedRoute` wrapper component in React Router. It reads `user` and `loading` states from `AuthContext`. If `user` exists, it renders the layout. If `user` is null, it redirects to `/login` using `<Navigate replace />`.

Q9: How does the backend prevent unauthorized actions?

Model Answer: In the `DELETE /api/posts/:id` route, the `authenticate` middleware verifies the JWT token and attaches `req.user`. We check if `post.author.toString() === req.user.id`. If they do not match, the server returns an HTTP 403 Forbidden status code.

● Section 4: Performance, Troubleshooting & Best Practices

Q10: How did you fix the blank white screen error during development?

Model Answer: A blank screen indicates an uncaught React runtime error. Backend middleware code (`app.use(cors(...))`) was accidentally placed in the client `api.js` file, causing a `ReferenceError`. I refactored `api.js` into a clean Axios instance with `baseUrl: 'http://localhost:4000/api'`.

Q11: How did you solve the Nodemon infinite restart loop?

Model Answer: Nodemon watched all files in the server directory. Since the backend wrote to `db.json` on startup, Nodemon detected file changes and triggered a restart loop. I resolved this by adding a `nodemon.json` config file explicitly ignoring `db.json` and `uploads/`.

Q12: How would you scale this application for 1,000,000 users?

Model Answer: 1) Add compound MongoDB indexes on `{ author: 1, createdAt: -1 }`. 2) Implement Redis caching for user feeds. 3) Offload image uploads to AWS S3/Cloudinary. 4) Implement cursor-based pagination.

✦ Summary Checklist Before Presentation

- ✓ Backend running on `http://localhost:4000`
- ✓ Frontend running on `http://localhost:5173`
- ✓ MongoDB Atlas connected (`pulse_social`)
- ✓ Demo credentials ready (`alex@example.com` / `password123`)
- ✓ Image upload pipeline tested and working